

Autonomous Multi-Goal Delivery Robot with Dynamic Replanning

COMPSCI-603: Robotics

Smitha Maganti | Khushei Meghana Meda | Akarsh Gupta | Skanda Chandrashekar

MOTIVATION & PROBLEM STATEMENT

Problem: In hospital settings, nurses spend significant time physically transporting medications, lab samples, and supplies between pharmacy and patient rooms.

Our Goal: Build a proof-of-concept autonomous delivery robot that can:

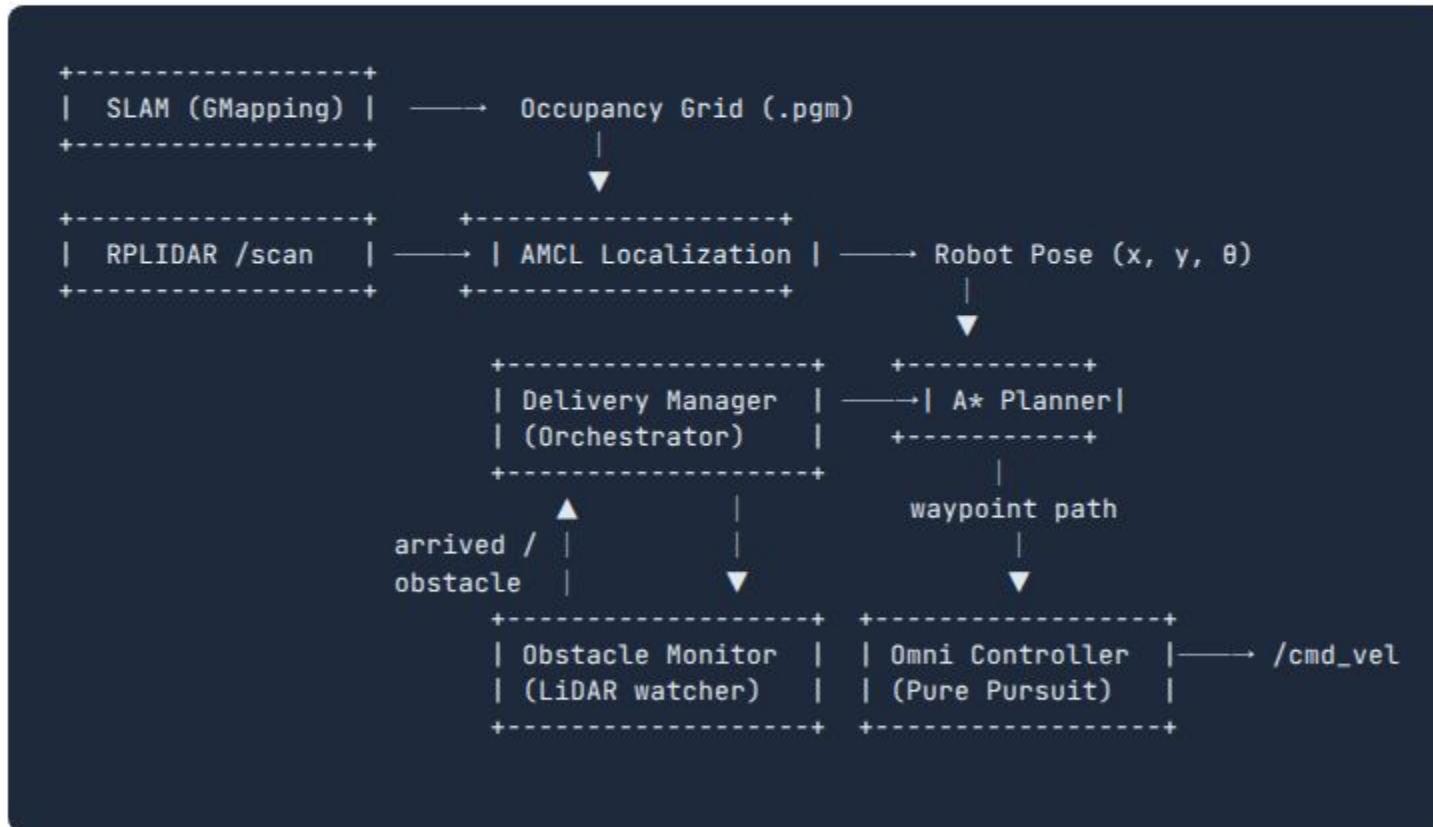
- Navigate a mapped indoor environment
- Visit N designated delivery locations in an efficient order
- Dynamically replan when unexpected obstacles block its path

Real-world parallel: Aethon TUG system (deployed in 600+ U.S. hospitals)

Course relevance: Directly applies SLAM, localization, A* search, occupancy grids, and motion control.



SYSTEM ARCHITECTURE OVERVIEW



Modular ROS Design

All modules are independent ROS nodes communicating via topics and services.

- **astar_planner:**
Service-based planning
- **omni_controller:**
Holonomic following
- **obstacle_monitor:** LiDAR scan watcher
- **delivery_manager:**
Top-level orchestrator

| SENSORS & PLATFORM

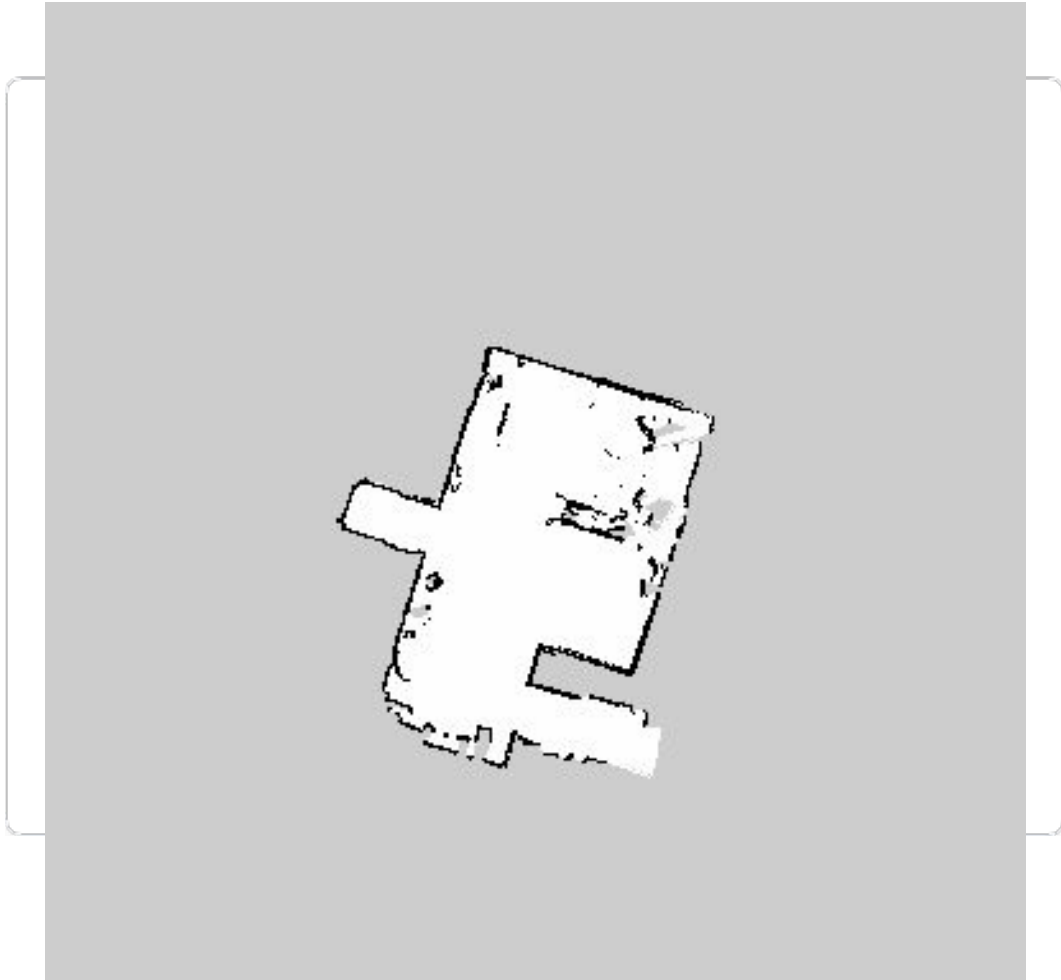
Platform: Triton Robot — omnidirectional (holonomic) base, NVIDIA Jetson Nano, ROS Noetic

Sensor	ROS Topic	Purpose
RPLIDAR A2M12	/scan	SLAM, Localization, Obstacles
Wheel Encoders	/odom	Odometry for AMCL motion

Note: Intel RealSense D435 camera was available but not required for 2D navigation.



O1: MAP BUILDING WITH SLAM



Method: GMapping

Process:

- 1 Teleop robot through environment using keyboard
- 2 Fuse LiDAR scans + odometry to build grid
- 3 Output: house_final.pgm + house_final.yaml

Occupancy Grid Properties:

- Cells: 0 = free, 100 = occupied, -1 = unknown
- Resolution: 5 cm/pixel
- Requirement: Map must match environment; furniture changes require re-SLAM.

O2A: What is A*

Evaluation function — pick the node with minimum $f(n)$

$$f(n) = g(n) + h(n)$$

$g(n)$ — cost so far

Actual distance from start to n

$$g(n) = g(n') + c(n', n)$$

$c = 1.0$ for cardinal moves

$c = \sqrt{2} \approx 1.414$ for diagonal moves

$h(n)$ — heuristic

Estimated distance from n to goal

$$h(n) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2}$$

Euclidean distance — admissible
(never overestimates true cost)

Optimality guarantee

$$h(n) \leq h^*(n) \quad \forall n$$

Heuristic $\leq$ true cost-to-go $\Rightarrow$ A* returns the shortest path

World $\leftrightarrow$ grid conversion

Map cell indices to metric coordinates and back

$$x_{\text{grid}} = \lfloor (x_{\text{world}} - ox) / r \rfloor$$

$$y_{\text{grid}} = \lfloor (y_{\text{world}} - oy) / r \rfloor$$

$$x_{\text{world}} = x_{\text{grid}} \cdot r + ox + r/2$$

$$y_{\text{world}} = y_{\text{grid}} \cdot r + oy + r/2$$

r = map resolution (ox, oy) = origin offset

O2B: A* PATH PLANNING

Obstacle Inflation

Before planning, expand occupied cells by robot's radius ($r = 0.22\text{m}$) to treat robot as a point.

```
for each occupied cell (cx, cy):  
    for each cell (dx, dy) within radius r:  
        if (dx - cx)^2 + (dy - cy)^2 <= r^2:  
            mark (dx, dy) as occupied
```



A* Search Logic

1. Inputs to A* Planner

The robot's current pose (AMCL) and target goal pose from navigation stack or user.

2. Occupancy Grid Map

Environment representation where:

- Free space = 0
- Occupied cells = Obstacles

3. Path Planning Logic

Computes the optimal traversable path considering walls and obstacles, not just Euclidean distance.

4. Output and Purpose

A sequence of waypoints enabling safe, efficient, and obstacle-free navigation.



03: Ordering Strategies - N Points Implementation

1. Fixed Order

Visit goals in launch file order. No computation, but inefficient.



2. Greedy Nearest Neighbor

Pick goal with min A* path cost from current position. Reactive.



Two-Ring Stopping System:

Since our robot glides holonomically directly toward targets, forcing it to hit an exact 0.00 meter coordinate causes endless jitter due to natural sensor noise. To fix this, we implemented a two-ring stopping system:

- **Outer Ring (40cm):** Applies proportional braking to smoothly bleed off momentum.
- **Inner Tolerance (15cm):** Controller instantly cuts motors, parking safely without jittering.

04A: Dynamic Replanning - Obstacle Detection

Our perception layer processes 360° LiDAR data in real-time. To efficiently identify genuine threats without freezing the CPU, we pass the raw data through two conceptual filters:

Filter 1: The Danger Circle

The LiDAR returns hundreds of data points per scan.

- Instantly discard anything further than 0.55 meters away.
- Isolate a small, manageable cluster of points inside the immediate "danger bubble".
- Convert points into absolute world coordinates.



Filter 2: The Path Corridor

Objects are threats only if directly in the path.

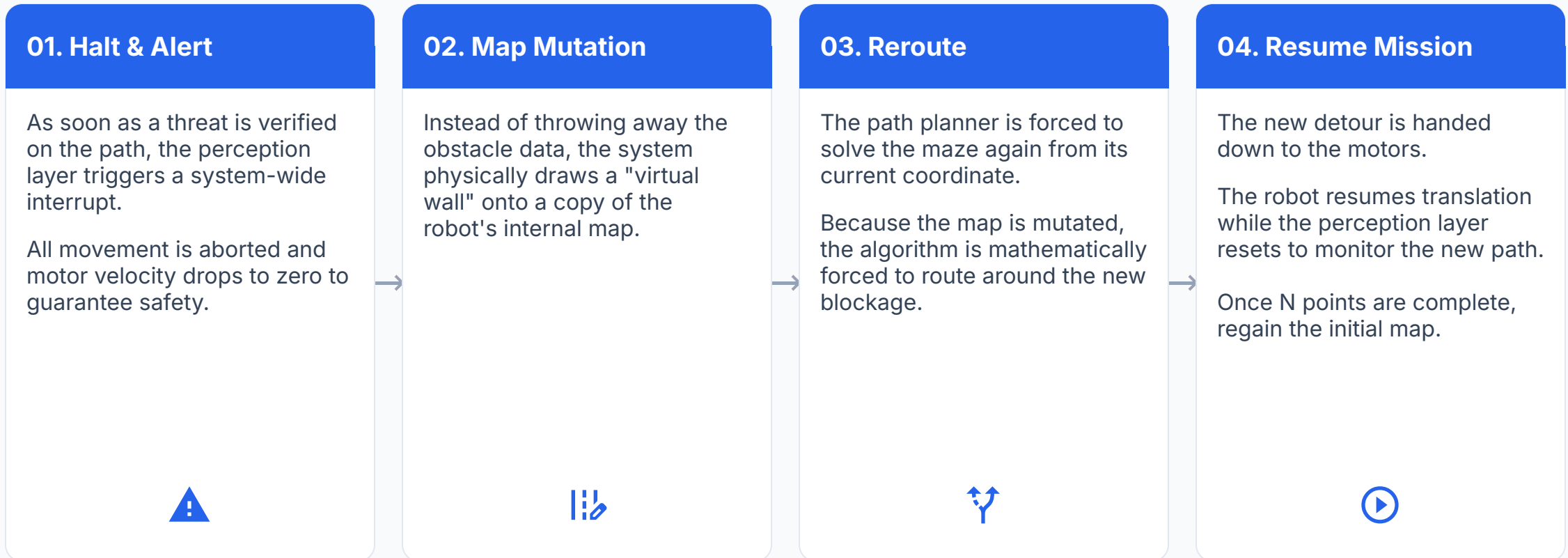
- Project a 0.22-meter safety corridor along the upcoming trajectory.
- Mathematical intersection with the Danger Bubble blocks the route.
- Triggers a halt and recalculates A* to find an alternative free path.



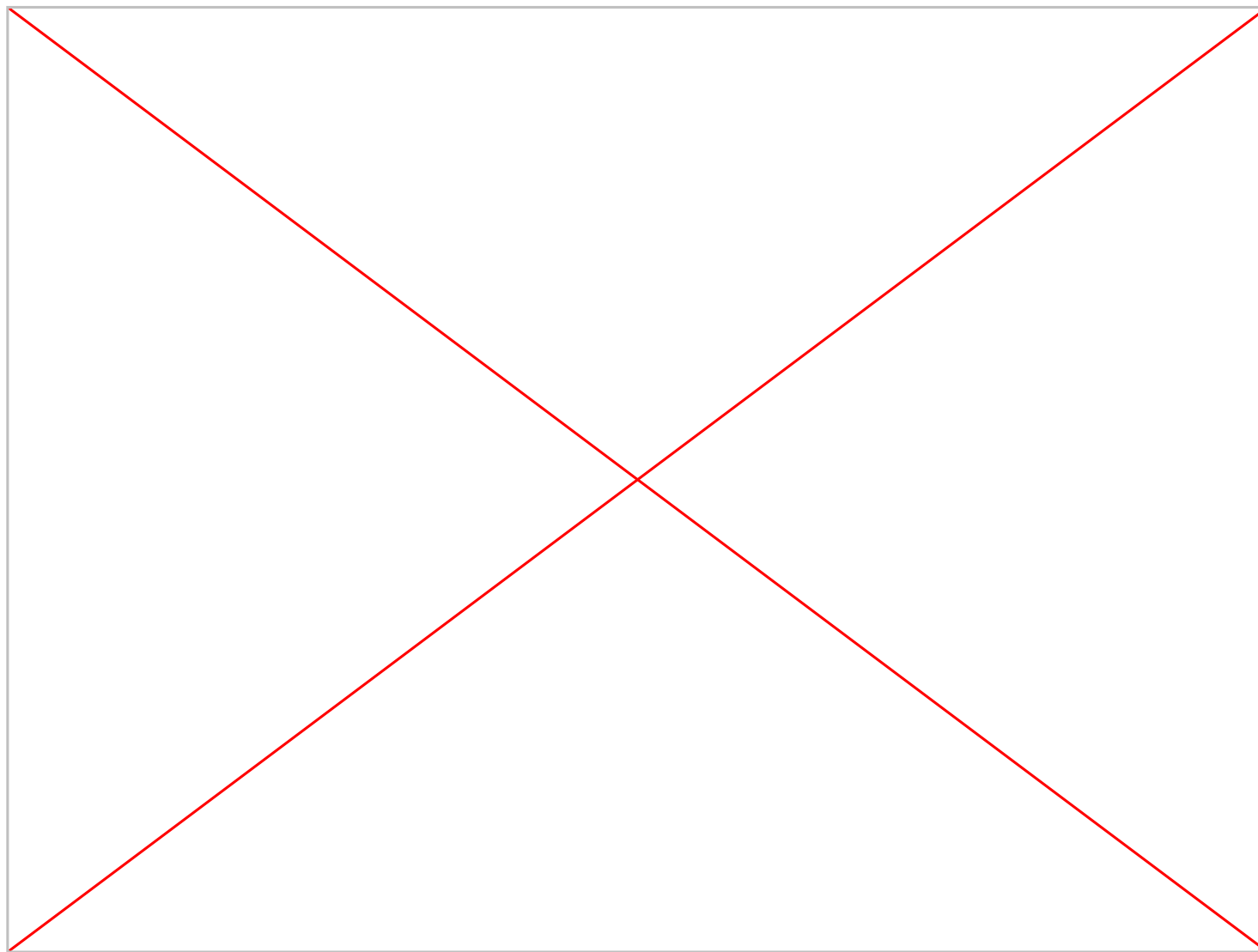
The Challenge: Memoryless Replanning

A* only knew about the original static map. Without physically adding new obstacles, the algorithm blindly generated the same path into the obstacle, causing an endless oscillation cycle: **Detect → Stop → Replan Same Route → Move → Detect.**

04B: Dynamic Replanning Flow (Work In Progress)



DEMO



| CHALLENGES FACED

1. Perceptual aliasing

Perceptual aliasing happens when two different places produce very similar sensor observations, so the robot incorrectly thinks they are the same location.

2. Map Invalidation

Environment changes broke AMCL localization. Lessons: In real deployments, maps need periodic refresh.

3. Robot kidnapping problem

Localization issue – robot suddenly finds itself in an unknown location without realizing it was moved there.

4. Mismatch b/w teleop and autonomous drive

The robot successfully crosses inter-room floor transitions under manual control, but autonomous navigation fails in these scenarios.

| CONCLUSION & FUTURE WORK

What we built:

End-to-end autonomous delivery system on a physical robot with custom A* planner, omni follower, and greedy-based orchestrator.

Future work:

- Multi floor delivery (use YOLO)
- Obstacle detection improvement
- Showcase the robot carrying and delivering medicines
- Delivery tracking and notification system
- Multi-robot coordination
- Emergency stops